UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM

APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 7 – Form Validation & Local Storage

Nama Mahasiswa	:	Nanik Wijayanti
NIM	:	23051430006
Kelas	:	A2
Tanggal Praktikum	:	03 April 2026
Dosen Pengampu	:	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 7 (Pertemuan 7)
Topik Utama	Form Validation & Local Storage
Sub-Topik	Validasi form tingkat lanjut dan penyimpanan data persisten di browser (Client-side Storage).
Durasi Praktikum	340 Menit
Tanggal Praktikum	3 April 2026
Nama Mahasiswa	Nanik Wijayanti
NIM	23051430006
Kelas	A2

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Menerapkan validasi form sebelum data dikirim untuk mencegah data kotor.
2	Memahami cara kerja Web Storage API (LocalStorage).
3	Mampu menyimpan, mengambil, mengupdate, dan menghapus (CRUD sederhana) data di LocalStorage.
4	Membuat aplikasi "Buku Harian Produksi" sederhana yang datanya tidak hilang saat browser direfresh.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)

4	Console Browser	Browser Modern	Untuk melakukan pengujian logika JavaScript, debugging, serta menampilkan hasil eksekusi kode secara langsung selama praktikum
5	Live Server	Ekstensi VS Code	Ekstensi untuk menjalankan server lokal agar perubahan kode web dapat terlihat secara *real-time* di browser

D. DASAR TEORI

Tuliskan ringkasan konsep/teori yang menjadi landasan pada modul ini. Ditulis dengan bahasa sendiri, bukan salinan mentah dari modul. Minimal 3 paragraf.

Pertemuan ini menitikberatkan pada konsep persistensi data di sisi klien agar aplikasi web kita tidak "pelupa". Secara teoritis, aplikasi web standar akan kehilangan semua datanya saat halaman dimuat ulang (*refresh*) karena data tersebut hanya tersimpan di memori sementara. Sebagai solusinya, kita menggunakan Web Storage API, khususnya LocalStorage, yang memungkinkan browser menyimpan data secara permanen dalam format pasangan kunci-nilai (*key-value pairs*). Kapasitas penyimpanan ini berkisar antara 5-10 MB, yang sangat ideal untuk membangun prototipe aplikasi industri yang bersifat *offline-first* atau sistem pencatatan laporan tanpa harus selalu terhubung ke basis data server yang rumit.

Landasan teori penting lainnya adalah mengenai integritas data melalui Validasi Formulir tingkat lanjut. Sebelum data operasional, seperti jumlah produksi atau nama operator, diizinkan masuk ke dalam penyimpanan, sistem harus bertindak sebagai "satpam" untuk mencegah masuknya data yang tidak logis atau "kotor". Dalam konteks Teknik Industri, validasi ini sangat krusial untuk mencegah *human error*, misalnya dengan memastikan jumlah produksi tidak bernilai negatif melalui logika JavaScript. Dengan memastikan data yang diinput adalah data yang bersih dan valid, laporan produksi yang dihasilkan akan tetap akurat dan dapat dipercaya untuk pengambilan keputusan.

Terakhir, modul ini mendasari operasionalnya pada transformasi data menggunakan format JSON (*JavaScript Object Notation*). Karena LocalStorage memiliki batasan teknis yang hanya mampu menyimpan data dalam bentuk teks (*string*), maka struktur data yang kompleks seperti *Object* atau *Array* tidak bisa disimpan secara langsung. Di sinilah kita menerapkan teknik konversi menggunakan JSON.stringify untuk membungkus objek menjadi teks saat disimpan, dan JSON.parse untuk membukanya kembali menjadi objek saat data diambil untuk ditampilkan di tabel. Pemahaman tentang alur data ini sangat penting untuk menjalankan operasi CRUD sederhana (*Create, Read, Update, Delete*) secara mandiri dan sistematis di dalam browser

D.1 Konsep Utama

Fokus utama pada pertemuan 7 ini adalah bagaimana cara memberikan "ingatan" atau persistensi data pada halaman web kita supaya tidak pelupa. Biasanya, ketika mengisi data di web lalu menekan tombol *refresh*, semua data itu akan langsung hilang. Pada pertemuan 7 ini, kita belajar menggunakan **LocalStorage**, yaitu semacam "laci penyimpanan" kecil di dalam browser pengguna yang bisa menampung data (sekitar 5-10 MB) secara permanen. Jadi, meskipun browser ditutup atau komputer dimatikan, data seperti "Buku Harian Produksi" yang kita buat akan tetap tersimpan di sana.

Namun, sebelum data boleh masuk ke "laci penyimpanan" tersebut, ada konsep **Validasi Form** yang bertugas sebagai "satpam" atau filter. Jika seorang operator salah memasukkan jumlah produksi dengan angka minus; kalau tidak divalidasi, data "kotor" ini akan merusak laporan. Dengan validasi JavaScript, kita memastikan hanya data yang benar-benar bersih, lengkap, dan logis yang boleh lanjut ke proses penyimpanan,. Ini sangat penting di industri untuk menjaga integritas data laporan harian agar tetap akurat.

Konsep kunci terakhir yaitu **Transformasi Data JSON**. Di sini kita menggunakan trik "bungkus dan buka". Fungsi 'JSON.stringify' untuk membungkus data objek menjadi teks saat hendak disimpan, dan memakai 'JSON.parse' untuk membuka kembali teks tersebut menjadi objek yang bisa diolah saat ingin ditampilkan lagi ke dalam tabel riwayat produksi,. Dengan memahami alur ini, kita bisa membuat aplikasi yang benar-benar fungsional tanpa perlu bantuan database server yang rumit

-

D.2 Keterkaitan dengan Teknik Industri

Materi dalam modul ini sangat relevan bagi mahasiswa Teknik Industri karena di era Industri 4.0, seorang insinyur tidak hanya dituntut untuk merancang sistem fisik, tetapi juga harus memastikan integritas dan persistensi data dalam sistem informasi seperti ERP atau MES. Pemrograman web pada tahap ini memberikan solusi praktis untuk mengelola data secara mandiri di sisi klien, yang sangat berguna untuk pembuatan prototipe aplikasi pelaporan lapangan yang bersifat *offline-first*. Dengan menguasai validasi dan penyimpanan lokal, kita bisa membangun sistem yang tidak hanya interaktif, tetapi juga memiliki "memori" untuk menyimpan rekam jejak operasional secara akurat.

Pada Pertemuan 7, konsep Form Validation dan Local Storage menjadi kunci utama untuk mentransformasi aplikasi monitoring menjadi sistem manajemen data yang andal dan terpercaya. Berikut adalah dua skenario aplikasi nyata dari materi Pertemuan 7 dalam dunia Teknik Industri:

1. Sistem *Buku Harian Produksi* Digital yang Persisten
   Dengan memanfaatkan LocalStorage, kita dapat membuat aplikasi pencatatan hasil produksi harian yang datanya tidak hilang meskipun browser di-*refresh* atau perangkat dimatikan sementara. Dalam lingkungan pabrik yang dinamis, kemampuan untuk menyimpan data sementara di memori browser (sekitar 5-10MB) tanpa perlu selalu terhubung ke database server pusat sangatlah krusial untuk menjaga kelancaran input data oleh operator di area yang minim sinyal. Penggunaan JSON stringify dan parse memastikan data produksi yang kompleks tetap terstruktur rapi di dalam "laci penyimpanan" digital tersebut.

2. Filter Integritas Data untuk Kendali Mutu (QC)
   Menggunakan JavaScript Validation, kita bisa membangun sistem "satpam" data yang mencegah masuknya informasi cacat atau tidak logis ke dalam laporan. Misalnya, saat operator menginput jumlah produksi, sistem secara otomatis menolak nilai nol atau negatif, serta memberikan peringatan *real-time* jika ada kolom wajib (seperti nama operator atau shift) yang terlewat. Hal ini sangat vital dalam Teknik Industri untuk menjaga integritas data laporan kualitas, sehingga keputusan manajemen yang diambil berdasarkan data tersebut tidak salah akibat *human error* saat pengisian formulir.

Secara keseluruhan, penguasaan materi ini memungkinkan kita mentransformasi aplikasi web sederhana menjadi alat kerja industri yang fungsional, yang mampu mengelola riwayat informasi (CRUD sederhana) secara mandiri dan sistematis

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

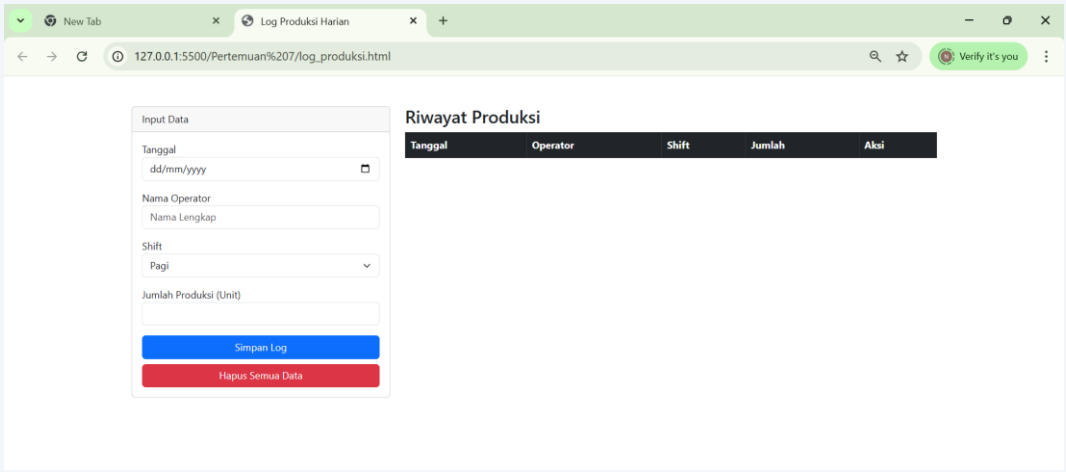
E.1 Langkah 1 – Membuat Form Input Produksi

Buat file baru `log_produksi.html`. Kita buat form sederhana.

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Log Produksi Harian</title>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">

    <div class="row">
        <!-- Kolom Kiri: Form Input -->
        <div class="col-md-4">
            <div class="card">
                <div class="card-header">Input Data</div>
                <div class="card-body">
                    <form id="formProduksi">
                        <div class="mb-3">
                            <label>Tanggal</label>
                            <input type="date" id="tanggal" class="form-
control"
required>
                        </div>
                        <div class="mb-3">
                            <label>Nama Operator</label>
                            <input type="text" id="operator" class="form-
control"
placeholder="Nama Lengkap" required>
                        </div>
                        <div class="mb-3">
                            <label>Shift</label>
                            <select id="shift" class="form-select" required>
                                <option value="Pagi">Pagi</option>
                                <option value="Siang">Siang</option>
                                <option value="Malam">Malam</option>
                            </select>
                        </div>
                        <div class="mb-3">
                            <label>Jumlah Produksi (Unit)</label>
```

```
                            <input type="number" id="jumlah" class="form-
control" min="1"
required>
                        </div>
                        <button type="submit" class="btn btn-primary w-
100">Simpan
Log</button>
                        <button type="button" id="btnHapusSemua" class="btn
btn-danger
w-100 mt-2">Hapus Semua Data</button>
                    </form>
                </div>
            </div>
        </div>

        <!-- Kolom Kanan: Tabel Data -->
        <div class="col-md-8">
            <h3>Riwayat Produksi</h3>
            <table class="table table-bordered table-striped">
                <thead class="table-dark">
                    <tr>
                        <th>Tanggal</th>
                        <th>Operator</th>
                        <th>Shift</th>
                        <th>Jumlah</th>
                        <th>Aksi</th>
                    </tr>
                </thead>
                <tbody id="tabelBody">
                    <!-- Data akan masuk di sini lewat JS -->
                </tbody>
            </table>
        </div>
    </div>

    <script src="script_log.js"></script>
</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – Tampilan Hasil Membuat Form Input Produksi

E.2 Langkah 2 – Logika JavaScript (Simpan & Baca)

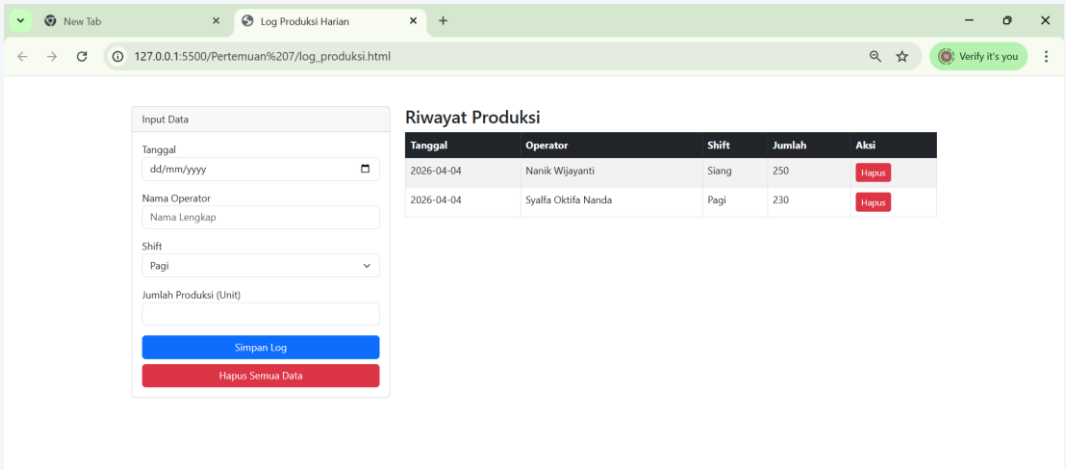
Buat file `script_log.js`.

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById("formProduksi");
const tabelBody = document.getElementById("tabelBody");
const btnHapusSemua = document.getElementById("btnHapusSemua");
// Kunci untuk LocalStorage
const STORAGE_KEY = "DATA_PRODUKSI_INDUSTRI";
// Fungsi Load Data saat halaman dibuka
document.addEventListener("DOMContentLoaded", function () {
  loadDataFromStorage();
});
// 2. Event Listener: Submit Form
formProduksi.addEventListener("submit", function (event) {
  event.preventDefault(); // Mencegah refresh halaman
  // Ambil Value dari Form
  const tanggal = document.getElementById("tanggal").value;
  const operator = document.getElementById("operator").value;
  const shift = document.getElementById("shift").value;
  const jumlah = document.getElementById("jumlah").value;
  // Validasi Sederhana (JavaScript)
  if (jumlah <= 0) {
    alert("Jumlah produksi harus lebih dari 0!");
    return;
  }
  // Buat Object Data
  const dataBaru = {
    id: Date.now(), // ID unik berdasarkan waktu
    tanggal: tanggal,
    operator: operator,
```

```
    shift: shift,
    jumlah: parseInt(jumlah),
  };
  // Simpan ke LocalStorage
  saveData(dataBaru);
  // Reset Form
  formProduksi.reset();
  // Refresh Tampilan Tabel
  loadDataFromStorage();
});
// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
  // Ambil data lama (jika ada), jika tidak ada array kosong
  let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  // Tambah data baru ke array
  dataLama.push(data);
  // Simpan kembali ke LocalStorage (Convert ke JSON String)
  localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}
// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
  // Ambil data
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  // Kosongkan tabel dulu
  tabelBody.innerHTML = "";

  // Loop data dan buat baris tabel
  data.forEach(function (item) {
    const row = document.createElement("tr");
    row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
            </td>
        `;
    tabelBody.appendChild(row);
  });
}

// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
```

```
  if (confirm("Yakin ingin menghapus log ini?")) {
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Filter data: hapus item yang id-nya cocok
    let dataBaru = data.filter((item) => item.id !== id);

    // Simpan ulang
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

    // Refresh tampilan
    loadDataFromStorage();
  }
};

// 6. Event Hapus Semua
btnHapusSemua.addEventListener("click", function () {
  if (confirm("PERINGATAN: Semua data akan dihapus permanen!")) {
    localStorage.removeItem(STORAGE_KEY);
    loadDataFromStorage();
  }
});
```

Screenshot Hasil:



Gambar E.2 – Tampilan Hasil Logika JS (Simpan & Baca)

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Filter Data

Pertanyaan / Instruksi Latihan:

> Tambahkan fitur filter atau pencarian. Buat input "Cari Nama Operator". Saat user mengetik, tabel hanya menampilkan data operator yang namanya cocok (Gunakan `filter` array).

Jawaban / Kode Solusi:

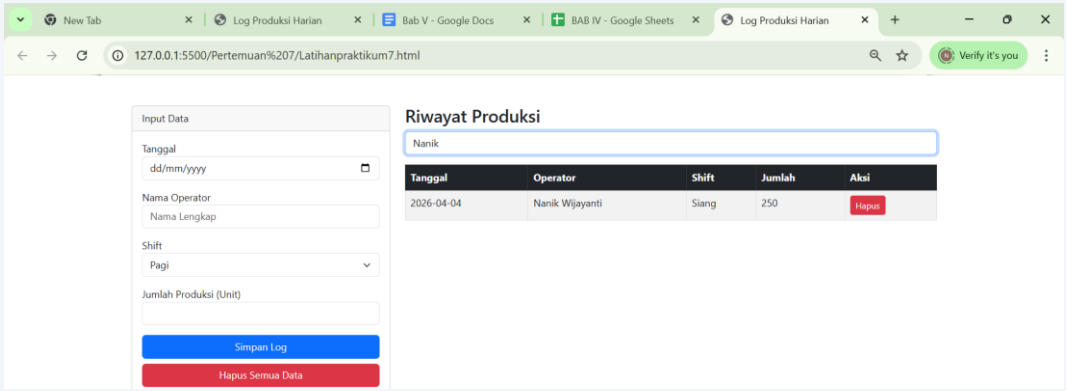
Menambahkan kode di HTML

```
<!—Latihan: Menambahkan Fungsi Filter/Pencarian Data-->
 <div class="mb-3">
          <input
            type="text"
            id="inputCari"
            class="form-control"
            placeholder="Cari Nama Operator..."
          />
        </div>
```

Menambahkan kode di JS:

```
<!--Menambahkan Fungsi Filter/Pencarian Data-->
 // LATIHAN 1: Fitur Pencarian Operator
const inputCari = document.getElementById("inputCari");

inputCari.addEventListener("input", function () {
  const keyword = inputCari.value.toLowerCase(); // Ubah ketikan jadi huruf
kecil
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  // Filter data berdasarkan nama operator yang diketik
  let dataTersaring = data.filter((item) =>
    item.operator.toLowerCase().includes(keyword),
  );

  // Kosongkan tabel dan isi dengan data hasil saringan
  tabelBody.innerHTML = "";
  dataTersaring.forEach(function (item) {
    const row = document.createElement("tr");
    row.innerHTML = `
      <td>${item.tanggal}</td>
      <td>${item.operator}</td>
      <td>${item.shift}</td>
      <td>${item.jumlah}</td>
      <td>
```

```
          <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
      </td>
    `;
    tabelBody.appendChild(row);
  });
});
```



Gambar F.1 – Tampilan Hasil Fitur Pencarian

F.2 Latihan 2 – Array Manipulation

Pertanyaan / Instruksi Latihan:

> Tambahkan tombol "Sortir berdasarkan Jumlah (Terbesar)" yang mengurutkan tabel dari jumlah produksi terbanyak ke terkecil.

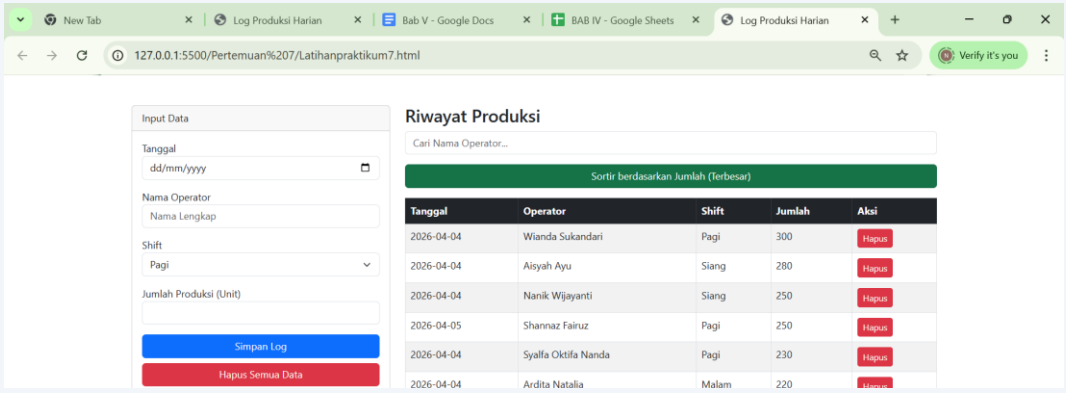
Jawaban / Kode Solusi:

Tambahkan kode pada HTML:

```
<!--Latihan 2: Menambahkan Fungsi Array Manipulation-->
        <div class="mb-3">
          <button id="btnSortir" class="btn btn-success w-100">
            Sortir berdasarkan Jumlah (Terbesar)
          </button>
        </div>
```

Tambahkan kode pada JS:

```
// LATIHAN 2: Fitur Sortir Jumlah Terbesar
const btnSortir = document.getElementById("btnSortir");

btnSortir.addEventListener("click", function () {
  // 1. Ambil data dari LocalStorage
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  // 2. Proses Sortir (Mengurutkan b ke a / Besar ke Kecil)
  data.sort((a, b) => b.jumlah - a.jumlah);

  // 3. Render ulang tabel dengan data yang sudah urut
  tabelBody.innerHTML = "";
  data.forEach(function (item) {
    const row = document.createElement("tr");
    row.innerHTML = `
      <td>${item.tanggal}</td>
      <td>${item.operator}</td>
      <td>${item.shift}</td>
      <td>${item.jumlah}</td>
      <td>
          <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
      </td>
    `;
    tabelBody.appendChild(row);
  });
});
```



Gambar F.2 – Tampilan Hasil Tombol Sortir

F.3 Tugas Proyek Mini – Aplikasi Checklist 5S

Deskripsi proyek mini:

Buat aplikasi checklist sederhana audit 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke).
a. Form berisi 5 checkbox (boolean) dan input nama auditor.
b. Saat disimpan, hitung skor (jumlah checklist dicentang / 5 * 100).
c. Simpan riwayat audit ke LocalStorage dan tampilkan di tabel (Tanggal, Auditor, Skor).

Penjelasan pendekatan/solusi yang digunakan:

[Tuliskan penjelasan Anda tentang cara menyelesaikan proyek mini ini]

Dalam menyelesaikan proyek mini aplikasi checklist audit 5S ini, saya berfokus pada penggabungan antara logika pemrograman JavaScript dengan tampilan antarmuka berbasis HTML agar menghasilkan sistem yang sederhana namun interaktif. Langkah pertama yang saya lakukan adalah merancang struktur data menggunakan Array of Objects. Setiap data audit disimpan dalam bentuk objek yang berisi atribut seperti *id, tanggal, shift, nama auditor, skor, serta hasil checklist (patuh dan kurang)*. Dengan pendekatan ini, data menjadi lebih terstruktur dan mudah untuk disimpan maupun ditampilkan kembali.

Selanjutnya, saya mengimplementasikan event handling pada form menggunakan event *submit*. Saat tombol simpan ditekan, sistem akan mengambil seluruh input seperti tanggal, shift, nama auditor, serta item checklist yang dicentang. Saya juga menambahkan validasi sederhana untuk memastikan semua input telah diisi sebelum data diproses. Dalam proses perhitungan, saya menggunakan konsep array dan filtering. Data checklist yang dicentang dimasukkan ke dalam array *patuh*, sedangkan item yang tidak dicentang diperoleh menggunakan metode *.filter()* untuk menghasilkan array *kurang*. Skor kemudian dihitung menggunakan rumus:

(jumlah checklist dicentang / 5 × 100)

Setelah data diproses, saya menyimpannya ke dalam LocalStorage agar data tetap tersimpan meskipun halaman di-refresh. Proses penyimpanan dilakukan dengan mengubah array menjadi format JSON menggunakan *JSON.stringify()*. Untuk menampilkan data, saya membuat fungsi khusus yaitu *loadDataAudit()* yang bertugas membaca data dari LocalStorage, melakukan filter berdasarkan nama auditor, dan menampilkannya ke dalam tabel menggunakan manipulasi DOM. Data ditampilkan secara dinamis dalam bentuk baris tabel lengkap dengan informasi tanggal, shift, auditor, skor, serta detail checklist.

Selain itu, saya juga menambahkan fitur pencarian (search/filter) menggunakan event *input*, sehingga pengguna dapat dengan mudah mencari data audit berdasarkan nama auditor. Fitur lain yang ditambahkan adalah tombol hapus, yang memungkinkan pengguna menghapus data tertentu dari LocalStorage menggunakan metode *.filter()*.

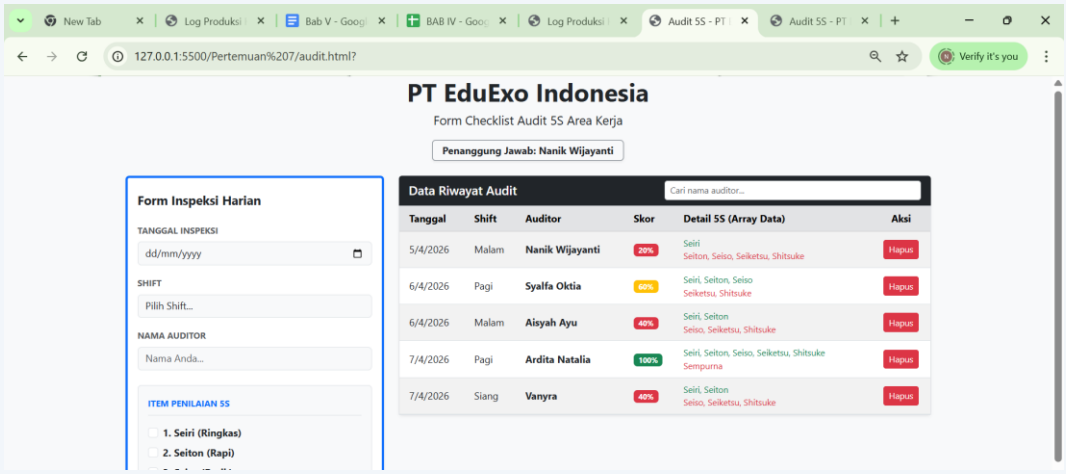
Dengan pendekatan ini, aplikasi dapat berjalan secara interaktif, mampu menyimpan data secara lokal, serta memberikan tampilan yang informatif dan mudah digunakan.

Kode di JS:

```
// 1. Inisialisasi Elemen
const formAudit = document.getElementById("formAudit");
const tabelBody5S = document.getElementById("tabelBody5S");
const inputCari = document.getElementById("inputCari");
const STORAGE_KEY = "DATA_AUDIT_5S";

// 2. Load awal
document.addEventListener("DOMContentLoaded", () => loadDataAudit());

// 3. FILTER / SEARCH
inputCari.addEventListener("input", function () {
  const kataKunci = inputCari.value.toLowerCase();
  loadDataAudit(kataKunci);
});

// 4. SIMPAN DATA
formAudit.addEventListener("submit", (e) => {
  e.preventDefault();

  // ambil input dengan BENAR (di dalam submit)
  const tanggalInput = document.getElementById("tanggalAudit").value;
  const shift = document.getElementById("shiftAudit").value;
  const nama = document.getElementById("namaAuditor").value.trim();

  // validasi
  if (!tanggalInput || !shift || !nama) {
    alert("Semua field wajib diisi!");
    return;
  }

  const dicentang = document.querySelectorAll(".ceklis-5s:checked");

  // ARRAY
  const master5S = ["Seiri", "Seiton", "Seiso", "Seiketsu", "Shitsuke"];
  let arrayPatuh = [];

  dicentang.forEach((box) => arrayPatuh.push(box.value));

  let arrayKurang = master5S.filter((item) => !arrayPatuh.includes(item));

  // format tanggal biar rapi
  const tanggalFormatted = new Date(tanggalInput).toLocaleDateString("id-
ID");

  const dataBaru = {
    id: Date.now(),
```

```
    tanggal: tanggalFormatted,
    shift: shift,
    auditor: nama,
    skor: (arrayPatuh.length / 5) * 100,
    patuh: arrayPatuh,
    kurang: arrayKurang,
  };

  saveData(dataBaru);
  formAudit.reset();
  loadDataAudit();
});

// 5. SIMPAN
function saveData(data) {
  let list = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  list.push(data);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(list));
}

// 6. RENDER
function loadDataAudit(filter = "") {
  let list = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  tabelBody5S.innerHTML = "";

  const dataTersaring = list.filter((item) =>
    (item.auditor || "").toLowerCase().includes(filter.trim()),
  );

  dataTersaring.forEach((item) => {
    const row = document.createElement("tr");

    const teksPatuh =
      Array.isArray(item.patuh) && item.patuh.length > 0
        ? item.patuh.join(", ")
        : "-";

    const teksKurang =
      Array.isArray(item.kurang) && item.kurang.length > 0
        ? item.kurang.join(", ")
        : "Sempurna";

    row.innerHTML = `
      <td class="ps-3">${item.tanggal}</td>
      <td>${item.shift || "-"}</td>
      <td><b>${item.auditor}</b></td>
```

```
      <td>
        <span class="badge ${
          item.skor === 100
            ? "bg-success"
            : item.skor >= 60
              ? "bg-warning"
              : "bg-danger"
        }">
          ${item.skor}%
        </span>
      </td>
      <td>
        <small class="text-success d-block">${teksPatuh}</small>
        <small class="text-danger d-block">${teksKurang}</small>
      </td>
      <td class="text-center">
        <button class="btn btn-sm btn-danger" onclick="hapus(${item.id})">
          Hapus
        </button>
      </td>
    `;

    tabelBody5S.appendChild(row);
  });
}

// 7. HAPUS
window.hapus = function (id) {
  if (confirm("Hapus data ini?")) {
    let list = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    localStorage.setItem(
      STORAGE_KEY,
      JSON.stringify(list.filter((i) => i.id !== id)),
    );
    loadDataAudit();
  }
};
```



Gambar F.3 – Tampilan Hasil Form Check List 5S

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Setelah menjalankan aplikasi checklist audit 5S, hasil yang ditampilkan sudah sesuai dengan materi pada pertemuan ini, terutama terkait validasi form dan penggunaan LocalStorage. Data yang diinput dapat diproses dan ditampilkan kembali di tabel tanpa error, sehingga menunjukkan bahwa program berjalan dengan baik.

Pada bagian form validation, sistem sudah mampu mengecek apakah input seperti tanggal, shift, dan nama auditor telah diisi. Jika ada yang kosong, data tidak akan disimpan. Hal ini sesuai dengan teori bahwa validasi diperlukan untuk mencegah data yang tidak lengkap masuk ke sistem.

Selanjutnya, penggunaan LocalStorage juga sudah sesuai teori. Data yang disimpan tidak hilang meskipun halaman direfresh. Hal ini menunjukkan bahwa LocalStorage dapat digunakan sebagai penyimpanan data sederhana di sisi client tanpa perlu database.

Dalam proses penyimpanan, saya menggunakan JSON.stringify() untuk mengubah object menjadi string, dan JSON.parse() untuk mengembalikannya menjadi object saat dibaca. Cara ini sesuai dengan teori karena LocalStorage hanya bisa menyimpan data dalam bentuk string.

Secara keseluruhan, aplikasi ini sudah menerapkan konsep dasar seperti validasi form, penyimpanan data menggunakan LocalStorage, serta pengolahan data dengan array dan object. Hasil yang diperoleh sudah sesuai dengan teori dan dapat berjalan dengan baik.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data Riwayat tidak muncul bahkan saya sudah submit dan mengisi form	Memastikan bahwa semua input (tanggal, shift, dan nama auditor) diambil di dalam event submit, bukan di luar. Selain itu, saya juga memastikan fungsi *loadDataAudit()* dipanggil setelah

	data disimpan agar tabel langsung terupdate. Saya juga mengecek LocalStorage di browser untuk memastikan data benar-benar tersimpan.
saya juga mengalami masalah pada input shift yang tidak muncul di tabel. Hal ini terjadi karena data shift belum dimasukkan ke dalam object saat proses penyimpanan.	Saya menambahkan properti *shift* ke dalam object data dan menampilkannya di bagian tabel.
Data tidak muncul setelah halaman di-refresh.	Mengecek bagian Console (F12) untuk melihat apakah ada error. Jika terdapat data yang rusak, maka LocalStorage dibersihkan, kemudian data diinput kembali dari awal di formnya.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan pemrograman pada modul ini sangat krusial untuk memastikan integritas dan persistensi data di lantai produksi. Dalam Teknik Industri, data yang "bersih" dan tersimpan dengan aman adalah fondasi dari pengambilan keputusan manajemen yang akurat. Berikut adalah tiga skenario konkret penerapan materi Pertemuan 7 di dunia industri:

1. Sistem "Buku Harian Produksi" yang Tangguh (Offline-First) Dalam skenario nyata di pabrik yang luas, koneksi internet seringkali tidak stabil. Dengan memanfaatkan LocalStorage, kita bisa membangun aplikasi pelaporan produksi di mana operator tetap bisa menginput data meskipun sinyal hilang. Data tidak akan hilang saat browser di-*refresh* karena tersimpan di memori browser pengguna. Penggunaan JSON stringify dan parse memastikan data produksi yang kompleks (seperti nama operator, shift, dan jumlah output) tetap terstruktur rapi dan siap dikirim ke server pusat saat koneksi kembali stabil.

2. Digital Poka-Yoke (Mencegah Human Error) Konsep Form Validation (Validasi Formulir) bertindak sebagai sistem *error-proofing* atau Poka-Yoke digital. Sebelum data masuk ke sistem ERP perusahaan, validasi JavaScript memastikan tidak ada data "kotor" yang terinput. Misalnya, sistem secara otomatis akan menolak jika operator salah memasukkan jumlah produksi bernilai negatif atau nol melalui logika if (jumlah <= 0). Hal ini sangat vital untuk menjaga akurasi laporan *Quality Control* dan mencegah kesalahan fatal dalam perhitungan stok inventori akibat salah ketik.

3. Aplikasi Audit 5S dan Checklist Digital Mandiri Teknik CRUD sederhana (Create, Read, Update, Delete) menggunakan LocalStorage memungkinkan kita membuat alat audit mandiri, seperti aplikasi checklist 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke). Setiap hasil audit dapat disimpan, ditampilkan dalam tabel riwayat secara *real-time*, bahkan dihapus jika terjadi kesalahan input tanpa perlu membangun infrastruktur *database* yang rumit. Ini memberikan fleksibilitas bagi seorang *Industrial Engineer* untuk membuat prototipe alat pemantauan lapangan yang cepat, murah, dan fungsional guna meningkatkan kedisiplinan kerja di area produksi.

Secara keseluruhan, penguasaan materi ini mengubah halaman web dari sekadar tampilan menjadi sistem informasi industri yang andal, yang mampu menjaga rekam jejak aktivitas produksi secara mandiri dan sistematis

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Saya belajar bahwa LocalStorage itu sangat berguna untuk memberikan "ingatan" pada halaman web agar data tidak hilang saat di-*refresh* atau browser ditutup. Di Teknik Industri, ini penting untuk membuat prototipe aplikasi *offline-first*, di mana kita bisa menyimpan "Buku Harian Produksi" sementara di dalam browser pengguna (sekitar 5-10 MB) tanpa harus selalu terhubung ke database server yang rumit
2	Penggunaan Validasi Form terbukti menjadi "satpam" yang menjaga integritas data kita. Saya belajar bahwa sebelum data disimpan, kita harus memastikan inputnya "bersih" dan logis, misalnya memastikan jumlah produksi tidak boleh nol atau negatif. Konsep ini sangat krusial di lapangan untuk mencegah *human error* yang bisa merusak laporan kualitas atau inventori.
3	Melalui JSON (Stringify & Parse), saya bisa melakukan konversi data agar bisa disimpan dengan aman. Karena LocalStorage hanya setia sama teks biasa (string), saya harus "membungkus" data objek yang kompleks menjadi teks pakai JSON.stringify saat mau disimpan, dan "membukanya" kembali jadi objek pakai JSON.parse saat datanya mau ditampilkan lagi ke tabel
4	Konsep CRUD Sederhana sangat efektif untuk membuat aplikasi yang benar-benar fungsional dan mandiri. Saya jadi tahu bagaimana cara mengelola data secara utuh, mulai dari membuat data baru (*Create*), menampilkannya di tabel (*Read*), sampai menghapus data tertentu berdasarkan ID unik (*Delete*) agar riwayat produksi tetap rapi dan akurat

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Nanik Wijayanti NIM: 23051430006	_____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm | Bawah: 3 cm | Kiri: 4 cm | Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → … → Laporan Modul 12 → Laporan Modul 14 & 15.

c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```
PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)
```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	**Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS**

— Selamat mengerjakan —